

# LC 1061: Lexicographically Smallest Equivalent String

---

*Authored by: Arpit Raj, LNMIIT Jaipur*

*Date: August 4, 2026*

*© All rights reserved.*

## Conceptual Walkthrough

**Given:** s1, s2, baseStr

**Let's understand via example:**

s1 = A A D Y A

s2 = T Y L L A

base = D O L L Y

**Then rules say:**

- A == T
- A == Y
- D == L
- Y == L
- A == A

*By transitive property, do union.*

→ **Groups are:** [A, T, Y, L, D] (All of them formed some group)

Now:

- If **D** of DOLLY is in any group, it picks up the smallest of the group, i.e., **A**.
- **O** is not in any group so it stays as it is.
- **L** and **Y** are in the group and their smallest is again **A**.

So, **A O A A A** → output

**NEW UNION RULE:**  
Instead of union by size, we use

`parent[max_char] = min_char`

## C++ Implementation

```

class dsu{
public:

    vector<int> parent;

    dsu(){
        parent.resize(26);

        for(int i=0;i<26;i++){
            parent[i]=i;
        }
    }

    int findparent(int node){

        if(parent[node]==node){
            return node;
        }

        parent[node]=findparent(parent[node]);
        return parent[node];
    }

    void unionit(int u,int v){
        int pu=findparent(u);
        int pv=findparent(v);

        if(pu==pv) return;

        if(pu<pv){
            parent[pv]=pu;
        }
        else{
            parent[pu]=pv;
        }
    }
};

class Solution {
public:
    string smallestEquivalentString(string s1, string s2, string baseStr) {

        int n=s1.size();
        dsu ds;
    }
};

```

```
for(int i=0;i<n;i++){
    int u=s1[i]-'a';
    int v=s2[i]-'a';
    ds.unionit(u,v);
}

for(int i=0;i<baseStr.size();i++){

    int boss=ds.findparent(baseStr[i]-'a');
    baseStr[i]=boss+'a';
}

return baseStr;

}

};
```